

Bài 6: QUAN HỆ KẾ THỪA GIỮA CÁC LỚP

Mục tiêu

Về kiến thức:

- Giúp sinh viên hiểu được mục đích và ý nghĩa của kế thừa, bao bọc thông tin và các phạm vi truy cập trong lớp.

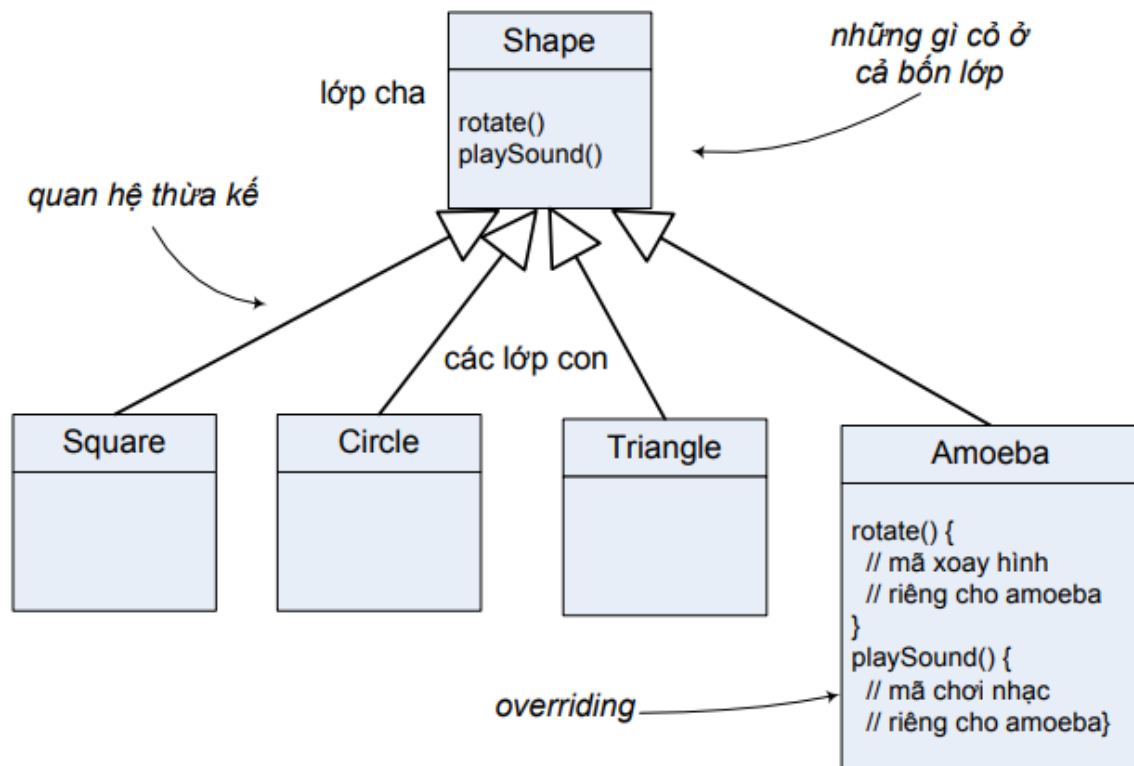
Về kỹ năng:

Giúp Sinh viên học cách vận dụng kế thừa và bao bọc thông tin để viết chương trình Java giải quyết các bài toán cụ thể và xác định ngữ cảnh sử dụng kế thừa.

6.1. Kế thừa giữa các lớp đối tượng

Giả sử Dâu xây dựng 4 lớp: Square (hình vuông), Circle (đường tròn), Triangle (hình tam giác), và Amoeba (hình trùng biến hình). Cả bốn đều là các hình với hai phương thức rotate() và playSound(). Do đó, anh ta dùng tư duy trừu tượng hóa để tách ra các đặc điểm chung và đưa chúng vào một lớp mới có tên Shape (hình nói chung). Sau đó, kết nối các lớp hình vẽ kia với lớp Shape bởi một quan hệ gọi là thừa kế.

Ta nói rằng "Square thừa kế từ Shape", "Circle thừa kế từ Shape", v.v.. Ta tháo gỡ rotate() và playSound ra khỏi 4 loại hình, và giờ thì chỉ còn phải quản lý một bản đặt tại lớp Shape. Shape được gọi là lớp cha (superclass) hay lớp cơ sở (base class) của bốn lớp kia. Còn bốn lớp đó là các lớp con (subclass) hay lớp dẫn xuất (derived class) của lớp Shape. Các lớp con thừa kế các phương thức của lớp cha. Nói cách khác, nếu lớp Shape có chức năng gì thì các lớp con của nó tự động có các chức năng đó.



Kế thừa trong java là một loại quan hệ giữa hai lớp (class) với nhau, trong đó có class cha (superclass) và class con (subclass). Khi kế thừa class con được sử dụng lại các phương thức và thuộc tính của class cha. Tuy nhiên, nó chỉ được kế thừa các thành phần public và protected của class cha. Nó không được phép truy cập đến thành phần private của class cha.

Tư tưởng của kế thừa trong java là có thể tạo ra một class mới được xây dựng trên các lớp đã xây dựng. Khi kế thừa từ một lớp ta có sử dụng lại các phương thức và thuộc tính của lớp cha, đồng thời có thể khai báo bổ sung thêm các phương thức và thuộc tính khác nhằm giới hạn phạm vi của đối tượng.

Khi ta dùng quan hệ thừa kế trong thiết kế, ta đặt các phần mã dùng chung tại một lớp và coi đó là lớp cha – lớp dùng chung trừu tượng hơn, các lớp cụ thể hơn là các lớp con. Các lớp con được thừa kế từ lớp cha đó. Quan hệ thừa kế có nghĩa rằng lớp con được thừa hưởng các thành viên (member) của lớp cha. Thành viên của một lớp là các biến thực thể và phương thức của lớp đó. Ví dụ, Shape trong ví dụ trên có hai thành viên `rotate()` và `playSound()`, Cow trong Hình 5.6 có các thành viên `name`, `age`, `getName()`, `getAge()`, `setName()`, `setAge()`.

Ta còn nói rằng lớp con chuyên biệt hóa (specialize) lớp cha. Nghĩa của "chuyên biệt hóa" ở đây gồm có hai phần: (1) lớp con là một loại con của lớp cha – thể hiện ở chỗ lớp con tự động thừa hưởng các thành viên của lớp cha, (2) lớp con có những đặc điểm của riêng nó - thể hiện ở chỗ lớp con có thể bổ sung các phương thức và biến thực thể mới của riêng mình, và nó có thể cài đè (override) các phương thức thừa kế từ lớp cha. Ví dụ, hình trùng biến hình (Amoeba) cũng là một hình (Shape), do đó lớp con Amoeba có tất cả những gì mà Shape có. Ngoài ra, Amoeba có thêm những đặc điểm riêng của thể loại hình trùng biến hình: các biến thực thể đại diện cho tâm xoay để phục vụ cách xoay của riêng nó, và nó định nghĩa lại các phương thức rotate để xoay theo cách riêng, định nghĩa lại playSound để chơi loại âm thanh riêng. Theo thuật ngữ, và cũng là từ khóa, của Java, lớp con "nối dài" (extends) lớp cha.

Các biến thực thể không bị cài đè vì việc đó là không cần thiết. Biến thực thể không quy định một hành vi đặc biệt nào và lớp con chỉ việc gán giá trị tùy chọn cho biến được thừa kế.

6.1.1. Cú pháp

Sử dụng từ khóa *extends* để kế thừa:

```
class Subclass-name extends Superclass-name {  
    //methods and fields  
}
```

Ví dụ:

```
class Employee {  
    float salary = 1000;  
}  
class Programmer extends Employee {  
    int bonus = 150;  
}  
public class InheritanceSample1 {  
    public static void main(String args[]) {  
        Programmer p = new Programmer();  
        System.out.println("Programmer salary is: " + p.salary);  
        System.out.println("Bonus of Programmer is: " + p.bonus);  
    }  
}
```

```
}
```

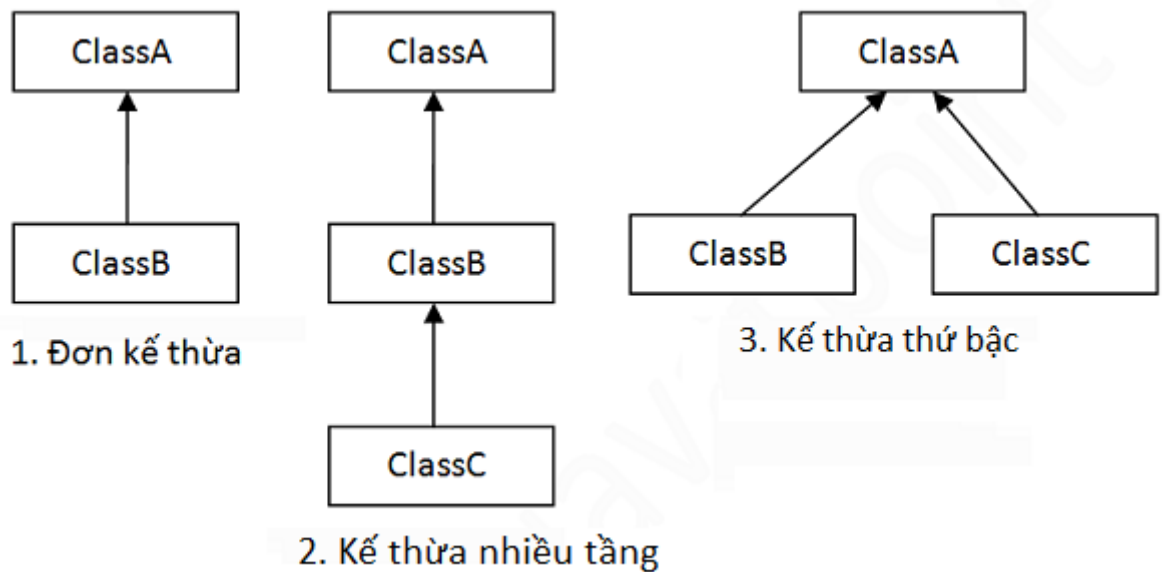
Kết quả:

Programmer salary is: 1000.0

Bonus of Programmer is: 150

6.1.2. Các kiểu kế thừa trong java

Có 3 kiểu kế thừa trong java đó là đơn kế thừa, kế thừa nhiều cấp, kế thừa thứ bậc. Khi một class được kế thừa từ nhiều class được gọi là đa kế thừa. Trong java, đa kế thừa chỉ được hỗ trợ thông qua giao diện(interface).



Chú ý: Đa kế thừa trong java không được support thông qua class.

Ví dụ:

```
class Animal {  
    void eat() {  
        System.out.println("eating...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("barking...");  
    }  
}  
  
public class TestInheritance1 {
```

```

    public static void main(String args[]) {
        Dog d = new Dog();
        d.bark();
        d.eat();
    }
}

```

Kết quả:

barking...

eating...

Ví dụ về kế thừa nhiều cấp

```

class Animal {
    void eat() {
        System.out.println("eating...");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("barking...");
    }
}

class BabyDog extends Dog {
    void weep() {
        System.out.println("weeping...");
    }
}

public class TestInheritance2 {
    public static void main(String args[]) {
        BabyDog d = new BabyDog();
        d.weep();
        d.bark();
        d.eat();
    }
}

```

```
}
```

Kết quả:

weeping...

barking...

eating...

Ví dụ về kế thừa thứ bậc

```
class Animal {  
    void eat() {  
        System.out.println("eating...");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("barking...");  
    }  
}  
  
class Cat extends Animal {  
    void meow() {  
        System.out.println("meowing...");  
    }  
}  
  
public class TestInheritance3 {  
    public static void main(String args[]) {  
        Cat c = new Cat();  
        c.meow();  
        c.eat();  
        // c.bark(); // compile error  
    }  
}
```

Kết quả:

meowing...

eating...

Để giảm thiểu sự phức tạp và đơn giản hóa ngôn ngữ, đa kế thừa không được support trong java. Hãy xem xét kịch bản sau: Có 3 lớp A, B, C. Trong đó lớp C kế thừa từ các lớp A và B. Nếu các lớp A và B có phương thức giống nhau và bạn gọi nó từ đối tượng của lớp con, như vậy khó có thể xác định được việc gọi phương thức của lớp A hay B.

```
class A {  
    void msg() {  
        System.out.println("Hello");  
    }  
}  
class B {  
    void msg() {  
        System.out.println("Welcome");  
    }  
}  
public class C extends A,B {  
    public static void main(String args[]) {  
        C obj = new C();  
        obj.msg();  
    }  
}
```

Kết quả:

Compile Time Error

6.2. Thiết kế cây thừa kế

Giả sử ta cần thiết kế một chương trình giả lập cho phép người dùng thả một đám các con động vật thuộc các loài khác nhau vào một môi trường để xem chuyện gì xảy ra. Ta hiện chưa phải viết mã mà mới chỉ ở giai đoạn thiết kế. Ta biết rằng mỗi con vật sẽ được đại diện bởi một đối tượng, và các đối tượng sẽ di chuyển loanh quanh trong môi trường, thực hiện các hành vi được lập trình cho loài vật đó. Ta được giao một danh sách các loài vật sẽ được đưa vào chương

trình: sư tử, hà mã, hổ, chó, mèo, sói. Và ta muốn rằng, khi cần, các lập trình viên khác cũng có thể bổ sung các loài vật mới vào chương trình.

Bước 1: ta xác định các đặc điểm chung và trừu tượng mà vật đều có. tất cả các loài động vật đều có. Các đặc điểm chung đó bao gồm năm thuộc tính:

picture - tên file ảnh đại diện cho con vật này

food – loại thức ăn mà con vật thích. Hiện giờ, biến này chỉ có hai giá trị: cỏ (grass) hoặc thịt (meat).

hunger – một biến int biểu diễn mức độ đói của con vật. Biến này thay đổi tùy theo khi nào con vật ăn và nó ăn bao nhiêu.

boundaries – các giá trị biểu diễn chiều dọc và chiều ngang (ví dụ 640 x 480) của khu vực mà các con vật sẽ đi lại hoạt động trong đó.

location – các tọa độ X và Y của con vật trong khu vực của nó.

và bốn phương thức:

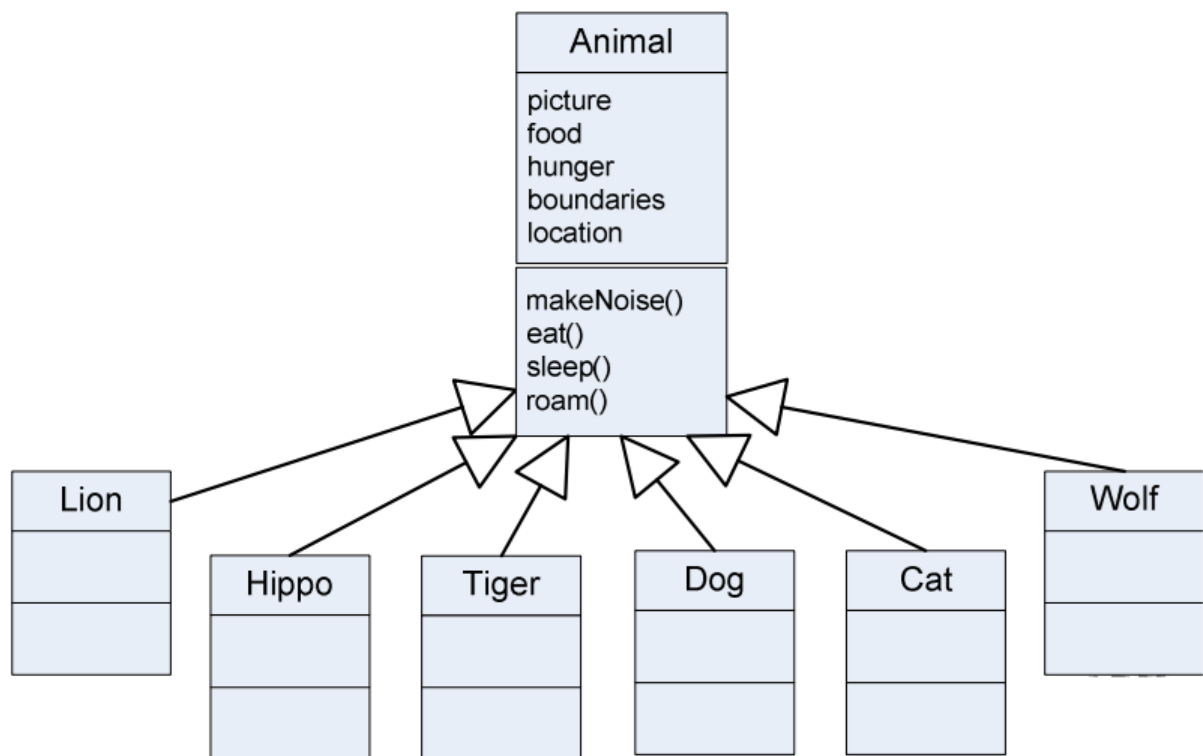
makeNoise() – hành vi khi con vật phát ra tiếng kêu

eat() – hành vi khi con vật gặp nguồn thức ăn ưa thích, thịt hoặc cỏ.

sleep() – hành vi khi con vật được coi là đang ngủ.

roam() – hành vi khi con vật không phải đang ăn hay đang ngủ, có thể chỉ đi lang thang đợi gặp món gì ăn được hoặc gặp biên giới lãnh địa.

Bước 2: Thiết kế một lớp với tất cả các thuộc tính và hành vi chung kể trên. Đây sẽ là lớp mà tất cả các lớp động vật đều có thể chuyên biệt hóa. Các đối tượng trong ứng dụng đều là các con vật (animal), do đó, ta sẽ gọi tên lớp cha chung của chúng là Animal. Ta đưa vào đó các phương thức và biến thực thể mà tất cả các con vật đều có thể cần. Kết quả là ta được lớp cha là lớp tổng quát hơn, hay nói cách khác là trừu tượng hơn, còn các lớp con mang tính đặc thù hơn, chuyên biệt hơn lớp cha.

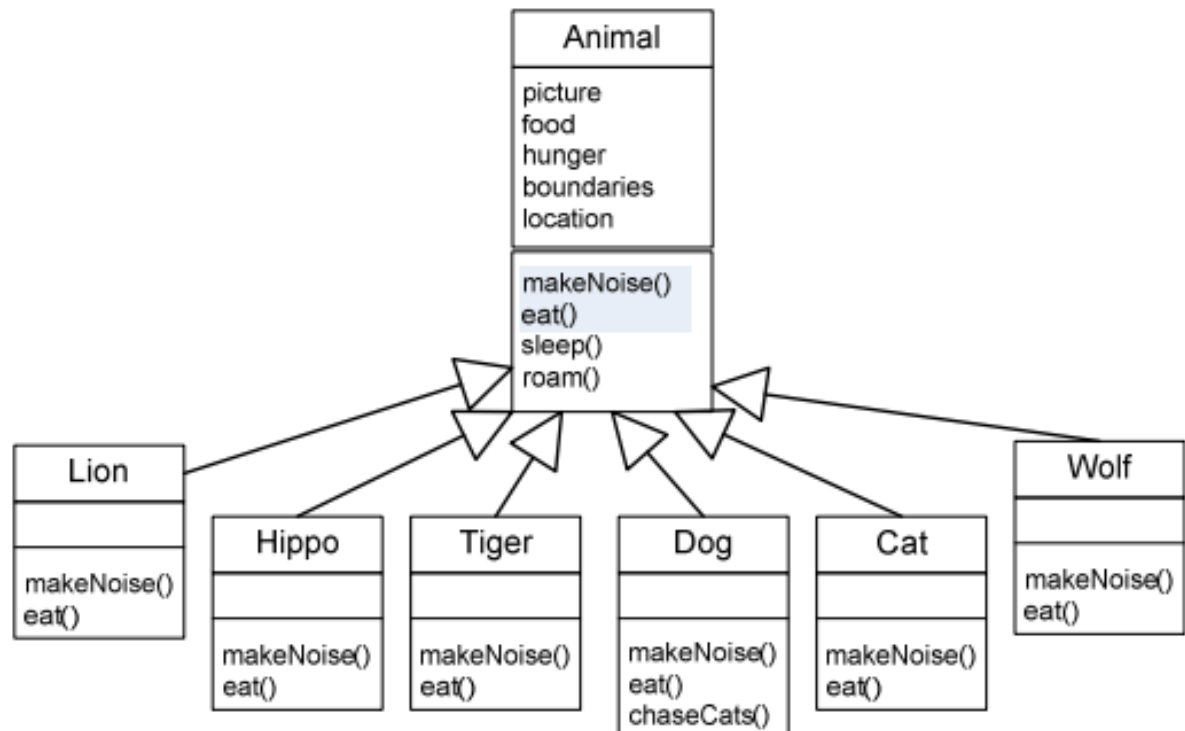


Ta đã biết rằng mỗi loại Animal đều có tất cả các biến thực thể đã khai báo cho Animal. Một con sư tử sẽ có các giá trị riêng cho picture, food, hunger, boundaries, và location. Một con hà mã sẽ có những giá trị khác cho bộ biến thực thể tương tự. Cũng như vậy đối với chó, hổ... Thế còn các hành vi của chúng thì sao?

Bước 3: Xác định xem các lớp con có cần các hành vi (cài đặt của các phương thức) đặc thù của thể loại con cụ thể đó hay không?

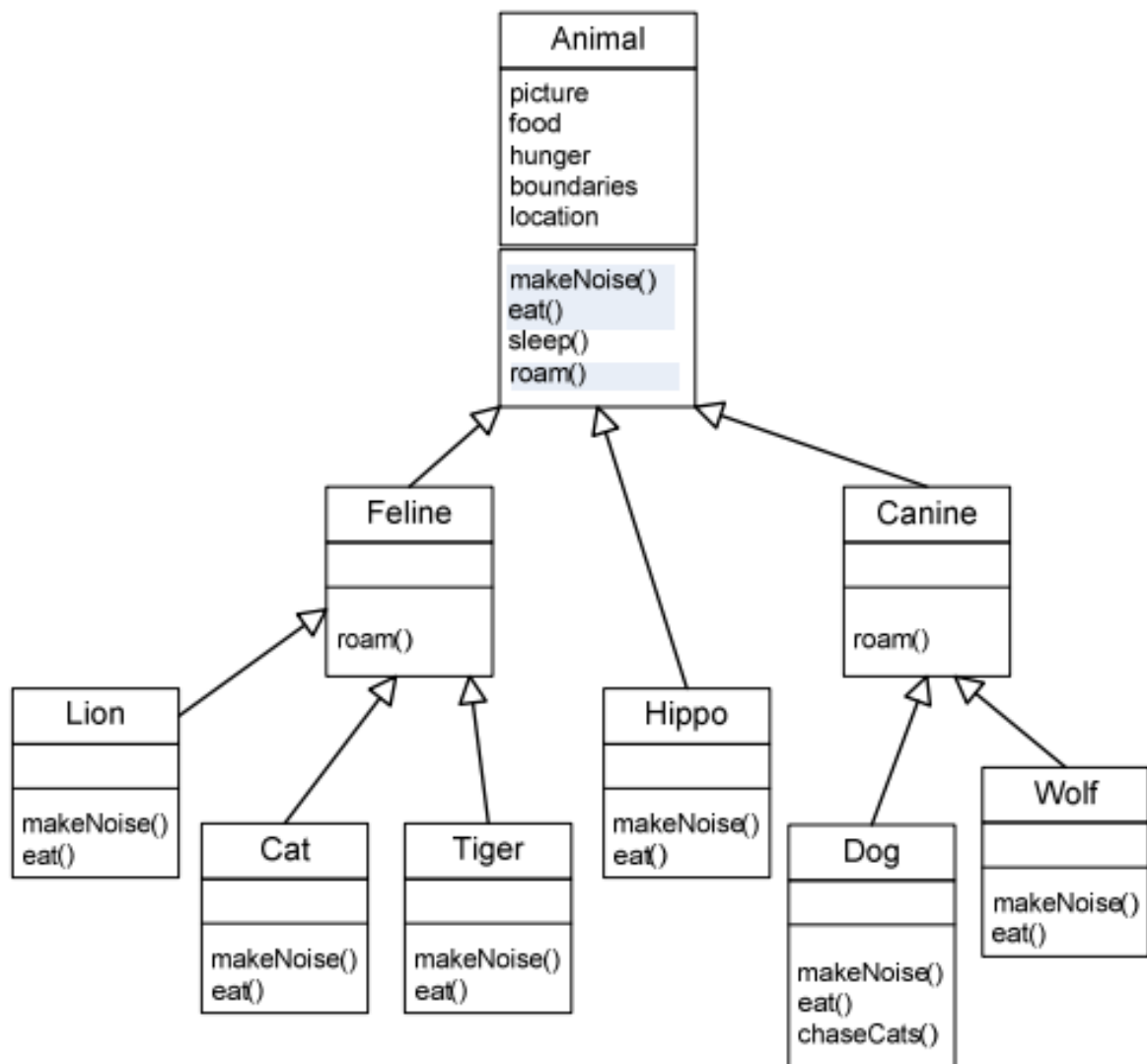
Để ý lớp Animal. Chắc chắn sư tử không ăn giống hà mã. Còn về tiếng kêu, ta có thể viết duy nhất một phương thức makeNoise tại Animal trong đó chơi một file âm thanh có tên là giá trị của một biến thực thể mà có giá trị khác nhau tùy loài, để con vật này kêu khác con vật khác. Nhưng làm vậy có vẻ chưa đủ vì tùy từng tình huống mà các loài khác nhau phát ra các tiếng kêu khác nhau, chẳng hạn tiếng kêu khi đang ăn và tiếng kêu khi gặp kẻ thù, v.v..

Do đó, ta quyết định rằng eat() và makeNoise() nên được cài đặt tại từng lớp con. Tạm coi các con vật sleep và roam như nhau và không cần cài đặt hai phương thức này. Ngoài ra, một số loài có những hành vi riêng đặc trưng của loài đó, chẳng hạn chó có thêm hành vi đuổi mèo (chaseCats()) bên cạnh các hành vi mà các loài động vật khác cũng có.



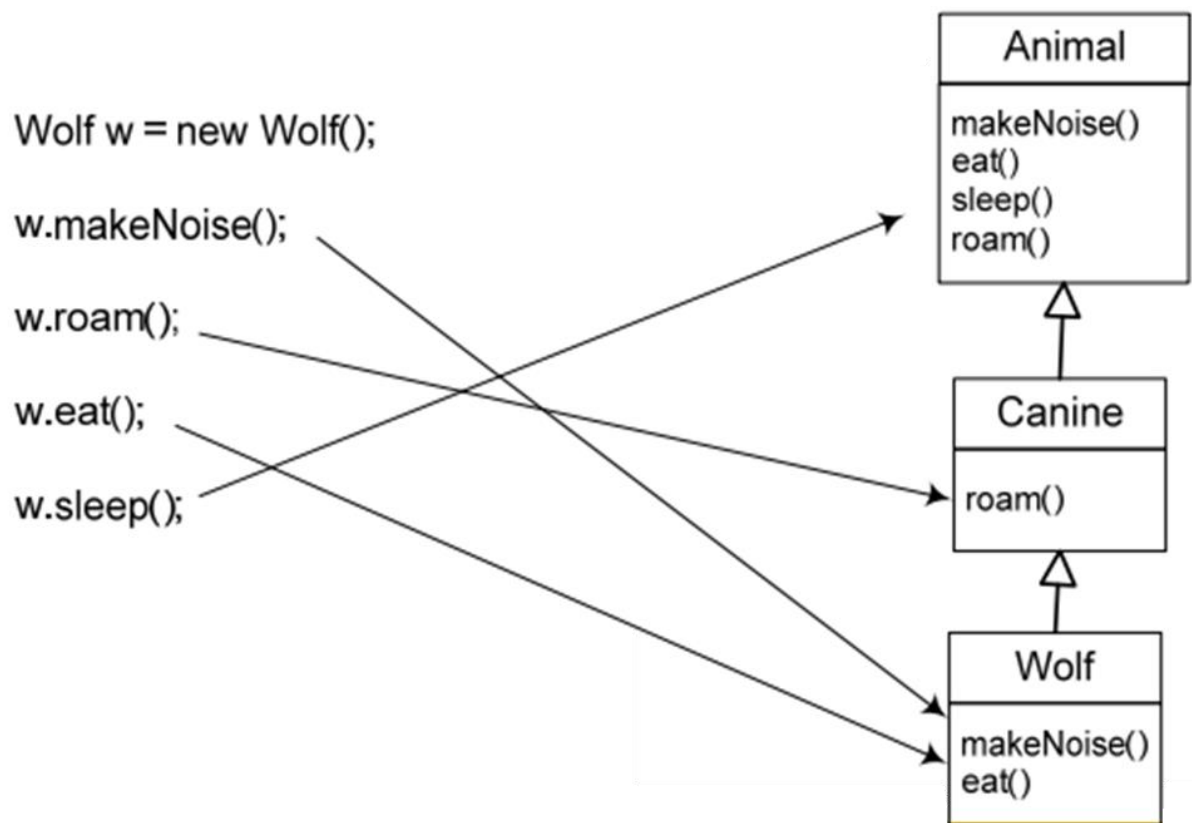
Bước 4: Tiếp tục dùng trừu tượng hóa tìm các lớp con có thể còn có hành vi giống nhau, với mục đích phân nhóm mịn hơn nếu cần.

Ví dụ, sói và chó có họ hàng gần, cùng thuộc họ Chó (canine) trong phân loại động vật học, chúng cùng có xu hướng di chuyển theo bầy đàn nên có thể dùng chung một phương thức `roam()`. Mèo, hổ và sư tử cùng thuộc họ Mèo (feline). Ba loài này có thể chung phương thức `roam()` vì khi di chuyển chúng cùng có xu hướng tránh đồng loại. Ta sẽ để cho hà mã tiếp tục dùng phương thức `roam()` tổng quát được thừa kế từ **Animal**.



6.3. Viết đề các phương thức

Lớp **Wolf** có bốn phương thức: `sleep()` được thừa kế từ **Animal**, `roam()` được thừa kế từ **Canine** (thực ra là phiên bản đề bản của **Animal**), và hai phương thức mà **Wolf** cài đề bản của **Animal** - `makeNoise()` và `eat()`. Khi ta tạo một đối tượng **Wolf** và gán một biến tham chiếu tới nó, ta có thể dùng biến đó để gọi cả bốn phương thức trên.



Khi gọi phương thức từ một tham chiếu đối tượng, ta đang gọi phiên bản đặc thù nhất của phương thức đó đối với lớp của đối tượng cụ thể đó. Nếu hình dung cây thừa kế theo kiểu các lớp cha ở phía trên còn các lớp con ở phía dưới, thì quy tắc ở đây là: phiên bản thấp nhất sẽ được gọi. Trong ví dụ dùng biến `w` để gọi phương thức cho một đối tượng `Wolf` ở trên, thứ tự từ thấp lên cao lần lượt là `Wolf`, `Canine`, `Animal`. Khi gọi một phương thức cho một đối tượng `Wolf`, máy ảo Java bắt đầu tìm từ lớp `Wolf` lên, nếu nó không tìm được một phiên bản của phương thức đó tại `Wolf` thì nó chuyển lên tìm tại lớp tiếp theo bên trên `Wolf` ở cây thừa kế, cứ như vậy cho đến khi tìm thấy một phiên bản khớp với lời gọi phương thức. Với ví dụ đang xét, như được minh họa trong hình vẽ, `w.makeNoise()` sẽ dẫn đến việc kích hoạt phiên bản của `Wolf`, `w.roam()` gọi phiên bản của `Canine`, v.v..

Một lớp con có thể viết đè, thay đổi nội dung thực hiện của những hàm thừa kế từ một lớp cha gọi là viết đè các hàm thành phần.

Khi những hàm này được gọi để thực thi đối với những đối tượng của lớp con thì nội dung thực hiện được định nghĩa ở lớp con sẽ được thực thi.

Ví dụ:

- Xây dựng lớp `Meo` có hàm thành phần `speak()`
- Xây dựng lớp `MeoCon` kế thừa lớp `Meo` và viết đè hàm `speak()`

- Gọi hàm speak() của lớp MeoCon
- Gọi hàm speak() của lớp Meo đối với đối tượng thuộc lớp MeoCon

```
class Meo{
    protected static void speak(){
        System.out.print("Meo Meo!"); }
}

class MeoCon extends Meo{
    protected static void speak(){
        System.out.print("Miu Miu!"); }
    public static void main(String []arg){
        Meo meo=new Meo();
        MeoCon meocon=new MeoCon();
        MeoCon.speak();
        System.out.println();
        meo.speak();
        System.out.println(); }
}
```

*** Lưu ý:**

- Định nghĩa mới của hàm viết đè phải có cùng định danh (tên gọi và danh sách tham biến) và cùng kiểu trả lại giá trị.
- Định nghĩa mới của hàm viết đè trong lớp con chỉ có thể xác định tất cả hoặc tập con các lớp ngoại lệ được kể ra trong mệnh đề cho qua ngoại lệ
- Các hàm final, static không được phép viết đè.
Phân biệt rõ hai cơ chế viết đè và nạp chồng là khác nhau trong Java.
- Viết đè yêu cầu cùng định danh hàm (cùng tên gọi, cùng danh sách tham số) và cùng kiểu trả lại đã được định nghĩa tại lớp cha.
- Nạp chồng yêu cầu khác nhau về định danh, nhưng giống nhau về tên gọi của hàm, vì thế chúng sẽ khác nhau về số lượng, kiểu, hay thứ tự của các tham biến.
- Hàm có thể nạp chồng ở trong cùng lớp hoặc ở các lớp con cháu.

Từ những lớp con khi muốn gọi tới các hàm ở lớp cha mà bị viết đè thì phải gọi qua toán tử đại diện cho lớp cha, đó là super(). Đối với hàm nạp chồng thì lại không cần

như thế. Lời gọi hàm nạp chồng được xác định thông qua danh sách các đối số hiện thời so sánh với đối số hình thức để xác định nội dung tương ứng.

Cơ chế che bóng của các biến

Về nguyên tắc, một lớp con không được phép viết đè các biến thành phần của lớp cha, nhưng có thể che khuất chúng tương tự như biến cục bộ trong lớp con.

Ví dụ *Viết đè và nạp chồng các hàm thành phần*

// KhachHang.java

```
class Den {  
    protected String loaiHoaDon = "Hoa don nho: "; // (1)  
    protected double docHoaDon(int giaDien) throws Exception { // (2)  
        double soGio = 10.0, hoaDonNho = giaDien * soGio;  
        System.out.println(loaiHoaDon + hoaDonNho);  
        return hoaDonNho;  
    }  
}  
  
class DenTuyp extends Den {  
    public String loaiHoaDon = "Hoa don lon: "; // Bị che bóng (3)  
    public double docHoaDon(int giaDien) throws Exception { // Viết đè hàm (4)  
        double soGio = 100.0;  
        double hoaDonLon = giaDien * soGio;  
        System.out.println(loaiHoaDon + hoaDonLon);  
        return hoaDonLon;  
    }  
    public double docHoaDon() {  
        System.out.println("Khong co hoa don!");  
        return 0.0;  
    }  
}  
  
public class KhachHang {  
    public static void main(String args[]) throws Exception { // (6)  
        DenTuyp den1 = new DenTuyp(); // (7)  
        Den den2 = den1; // (8)
```

```

        Den den3 = new Den(); // (9)
        // Gọi các hàm đã viết đề
        den1.docHoaDon(1000); // (10)
        den2.docHoaDon(1000); // (11)
        den3.docHoaDon(1000); // (12)
        // Truy nhập tới các biến thành phần đã bị viết đề (bị che bóng)
        System.out.println(den1.loaiHoaDon); // (13)
        System.out.println(den2.loaiHoaDon); // (14)
        System.out.println(den3.loaiHoaDon); // (15)
        // Gọi các hàm nạp chồng
        den1.docHoaDon();
    }
}

```

Kết quả thực hiện của chương trình KháchHang:

Hóa đơn lớn: 100000.00

Hóa đơn lớn: 100000.00

Hóa đơn nhỏ: 10000.00

Hóa đơn lớn:

Hóa đơn nhỏ:

Hóa đơn nhỏ:

Không có Hóa đơn!

6.4. Mối quan hệ IS-A và HAS-A

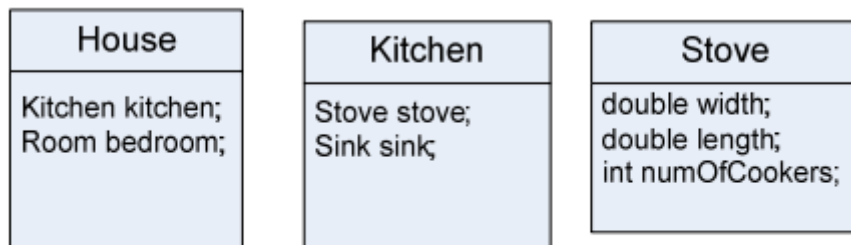
Khi một lớp kế thừa từ một lớp khác, ta nói rằng lớp con chuyên biệt hóa lớp cha. Nhưng liệu khi nào thì nên chuyên biệt hóa một lớp khác? Nhớ lại rằng lớp cha là loại tổng quát, còn lớp con là loại cụ thể và chuyên biệt, là loại con của lớp cha. Nhìn từ khía cạnh khác, tập hợp các đối tượng mà lớp con đại diện là một tập con của các đối tượng mà lớp cha đại diện. Do đó, để đưa ra lựa chọn đúng đắn cho vấn đề nên hay không nên để lớp X là lớp chuyên biệt hóa lớp Y, ta có một phương pháp hiệu quả: kiểm tra quan hệ IS-A, nghĩa là xem thứ này có là thứ kia hay không.

Để xem X có nên là lớp con của Y hay không, ta đặt câu hỏi theo dạng "Nếu phát biểu một cách tổng quát rằng loại X là một dạng/thứ/kiểu của loại Y thì có lý hay không?". Nếu câu trả lời là "Có", thì X có thể là lớp con của Y.

Ví dụ: Tam giác là một hình (Triangle IS-A Shape)? Đúng. Mèo là một động vật họ Mèo (Cat IS-A Feline)? Đúng. Xe tải là một phương tiện giao thông (Truck IS-A Vehicle)? Đúng. Nghĩa là, Triangle có thể là lớp con của Shape, Cat có thể là lớp con của Feline, Truck có thể là lớp con của Vehicle.

Ta xét tiếp: Phòng bếp là một cái nhà (Kitchen IS-A House)? Chắc chắn sai. Ngược lại thì sao? Nhà là một phòng bếp (House IS-A Kitchen)? Đúng là có một số người vì phong tục hay điều kiện sống mà ngôi nhà của họ chỉ có một phòng duy nhất nên đó vừa là nơi nấu bếp vừa là phòng cho nhiều chức năng khác. Tuy nhiên, các trường hợp đó chỉ là "một số", nên câu trả lời tổng quát vẫn là "Sai". Cho nên, Kitchen không thể là lớp con của House hay ngược lại.

Phòng bếp và nhà rõ ràng có liên quan đến nhau, nhưng không phải qua quan hệ thừa kế mà là một quan hệ chứa – HAS-A. Câu hỏi ở đây là: Nhà có chứa một phòng bếp hay không (House HAS-A Kitchen)? Nếu câu trả lời là "Có", điều đó có nghĩa House có một biến thực thể kiểu Kitchen. Nói cách khác, House có một tham chiếu tới một đối tượng Kitchen, chứ House không chuyên biệt hóa Kitchen hay ngược lại.



*House HAS-A Kitchen, Kitchen HAS-A Stove.
Nhà có một phòng bếp, phòng bếp có một cái bếp.
Nhưng không có lớp nào thừa kế một lớp khác.*

Quan hệ HAS-A trong Java được cài đặt bằng tham chiếu đặt tại đối tượng chứa chiếu tới đối tượng thành phần. Quan hệ HAS-A giữa hai lớp thể hiện một trong ba quan hệ: kết hợp (association), tụ hợp (aggregation) và hợp thành (composition) mà các tài liệu về thiết kế hướng đối tượng thường nói đến. Giữa hai lớp có quan hệ kết hợp nếu như các đối tượng thuộc lớp này cần biết đến đối tượng thuộc lớp kia để có thể thực hiện được công việc của mình. Chẳng hạn, một người nhân viên chịu sự quản lý của một người quản lý, ta có quan hệ kết hợp nối từ Employee tới Manager, thể hiện ở việc mỗi đối tượng Employee có một tham chiếu boss kiểu Manager. Hợp thành và tụ hợp là các

quan hệ giữa một đối tượng và thành phần của nó (cũng là đối tượng). Khác nhau ở chỗ, với quan hệ hợp thành, đối tượng thành phần là phần không thể thiếu được của đối tượng chứa nó, còn với quan hệ tụ hợp thì ngược lại. Ví dụ, một cuốn sách bao gồm nhiều trang sách và một cuốn sách không thể tồn tại nếu không có trang nào. Do đó giữa Book (sách) và Page (trang) có quan hệ hợp thành. Thư viện có nhiều sách, nhưng thư viện không có cuốn sách nào vẫn là một thư viện, nên quan hệ giữa Library (thư viện) và Book là quan hệ tụ hợp. Java không có cấu trúc nào dành riêng để cài đặt các quan hệ tụ hợp hay hợp thành. Ta chỉ cài đặt đơn giản bằng cách đặt vào đối tượng chủ các tham chiếu tới đối tượng thành phần, hay nói cách khác là phân rã thành các quan hệ HAS-A, chẳng hạn quan hệ hợp thành giữa Book và Page có thể được phân rã thành 'Book HAS-A ArrayList<Page>' và nhiều quan hệ 'ArrayList<Page> HAS-A Page'. Các ràng buộc khác được đảm bảo bởi các phương thức có nhiệm vụ khởi tạo hay sửa các tham chiếu đó.

Quay lại quan hệ IS-A, có một điểm cần lưu ý: quan hệ thừa kế IS-A chỉ có một chiều. Ví dụ: "Tam giác là một hình" là phát biểu có lý, nhưng khẳng định theo chiều ngược lại, "Hình là một tam giác", thì không đúng. Có nhiều hình là hình tam giác, nhưng cũng có vô số hình không phải hình tam giác.

Thực ra, lưu ý trên là hiển nhiên, nếu ta nhớ đến mô tả về lớp con tại mục trước: Lớp con chuyên biệt hóa lớp cha. Đến đây, chúng ta chưa kết thúc câu chuyện về quan hệ thừa kế. Chương sau sẽ tiếp tục trình bày về các vấn đề hướng đối tượng. Một số giải pháp thiết kế trong chương này sẽ được xem lại và cải tiến.

Khi nào nên dùng kế thừa:

- **NÊN** dùng quan hệ thừa kế khi một lớp là một loại cụ thể hơn của một lớp cha. Ví dụ, tài khoản tiết kiệm (saving account) là một loại tài khoản ngân hàng (bank account), nên SavingAccount là lớp con của BankAccount là hợp lý.
- **NÊN** cân nhắc việc thừa kế khi ta có một hành vi (mã đã được viết) nên được dùng chung giữa nhiều lớp thuộc cùng một kiểu tổng quát nào đó. Ví dụ, Square, Circle và Triangle trong bài toán của Đậu và Tuất cùng cần xoay và chơi nhạc, nên việc đặt các chức năng đó tại một lớp cha Shape là hợp lý. Tuy vậy, cần lưu ý rằng mặc dù thừa kế là một trong những đặc điểm quan trọng của lập trình hướng đối tượng nhưng nó không nhất thiết

là cách tốt nhất cho việc tái sử dụng hành vi. Quan hệ thừa kế giúp ta khởi động việc tái sử dụng, và nó thường là lựa chọn đúng khi thiết kế, nhưng các mẫu thiết kế sẽ giúp ta nhận ra những lựa chọn khác tinh tế và linh hoạt hơn.

- **KHÔNG NÊN** dùng thừa kế chỉ nhằm mục đích tái sử dụng mã của một lớp khác, trong khi quan hệ giữa lớp cha và lớp con vi phạm một trong hai quy tắc ở trên. Ví dụ, giả sử ta đã viết cho lớp DoorBell (chuông cửa) một đoạn mã dành riêng cho việc in, và giờ ta cần viết mã cho chức năng in của lớp Piano. Không nên vì nhu cầu đó mà cho Piano làm lớp con của DoorBell. Đàn piano không phải là một loại chuông gọi cửa. (Giải pháp nên chọn cho tình huống này là: phần mã cho chức năng in nên được đặt trong một lớp Printer, và các lớp cần có chức năng in sẽ hưởng lợi từ lớp Printer đó qua một quan hệ HAS-A.)
- **KHÔNG NÊN** dùng quan hệ thừa kế nếu lớp con và lớp cha không qua được thử nghiệm IS-A. Hãy tự kiểm tra xem lớp con có phải là một kiểu chuyên biệt của lớp cha hay không. Ví dụ: Bike IS-A Vehicle (xe đạp là một phương tiện giao thông) hợp lý. Nhưng Vehicle IS-A Bike (phương tiện giao thông là một loại xe đạp) thì không được.

Lợi ích của kế thừa:

- Lợi ích thứ nhất: tránh lặp các đoạn mã bị trùng lặp. Ta có thể loại bỏ được những đoạn mã trùng lặp bằng cách tách ra các hành vi chung của một nhóm các lớp đối tượng và đưa phần mã đó vào một lớp cha. Nhờ đó, khi ta cần sửa nó, ta chỉ cần cập nhật mã ở duy nhất một nơi, và sửa đổi đó có hiệu lực tại tất cả các lớp kế thừa hành vi đó. Công việc gói gọn trong việc sửa và dịch lớp cha. Tóm lại: **ta không phải động đến các lớp con!** Với ngôn ngữ Java, chương trình là một tập các lớp. Do đó, ta không cần phải dịch lại các lớp con để có thể dùng được phiên bản mới của lớp cha. Điều hỏi duy nhất là phiên bản mới của lớp cha không phá vỡ cái gì của lớp con. Nghĩa cụ thể của từ "phá vỡ" trong ngữ cảnh trên sẽ được trình bày chi tiết sau. Tạm thời, ta chỉ cần hiểu rằng hành động đó có nghĩa là sửa cái gì đó tại lớp cha mà lớp

con bị phụ thuộc vào, chẳng hạn như sửa kiểu tham số, hay kiểu trả về, hoặc tên của một phương thức nào đó.

- Lợi ích thứ hai: ta định nghĩa được một giao thức chung cho tập các lớp gắn kết với nhau bởi quan hệ thừa kế. Quan hệ thừa kế cho phép ta đảm bảo rằng tất cả các lớp con của một lớp đều có tất cả các phương thức mà lớp đó có. Đó là một dạng giao thức mà lớp đó tuyên bố với tất cả các phần mã khác rằng: "Tất cả các thể loại con của tôi (nghĩa là các lớp con) đều có thể làm những việc này, với các phương thức trông như thế này...". Nói cách khác, ta thiết lập một hợp đồng (contract).

Animal
makeNoise() eat() sleep() roam()

đây là tuyên bố rằng bất cứ Animal nào cũng có thể làm bốn việc đó. Nghĩa là các phương thức với các kiểu tham số và giá trị trả về đó

Lưu ý rằng, khi nói về Animal bất kì, ý ta đang nói về đối tượng Animal hay đối tượng thuộc bất cứ lớp nào có Animal là tổ tiên trong cây phả hệ. Khi ta định nghĩa một kiểu tổng quát (lớp cha) cho một nhóm các lớp, bất cứ lớp con nào trong nhóm đó đều có thể dùng thay cho vị trí của lớp cha. Ta đã có Wolf là một loại con của Animal; một đối tượng Wolf có tất cả các thành viên mà một đối tượng Animal có. Vậy thì lô-gic hiển nhiên: một đối tượng Wolf có thể được coi là thuộc loại Animal; nơi nào dùng được Animal thì cũng dùng được Wolf.

6.5. Toán tử this() và super()

Toán tử tạo lập this() Toán tử tạo lập này được sử dụng để tham chiếu đến các thành phần (là các thuộc tính và phương thức) của lớp hiện thời	Toán tử tạo lập super() Được sử dụng tham chiếu đến các thành phần thuộc lớp cha trực tiếp
<pre>class BongDen{ // Biến thành phần (1) private int soWatts; private boolean batTat;</pre>	<pre>class BongDen{ // Biến thành phần (1) private int soWatts; private boolean batTat;</pre>

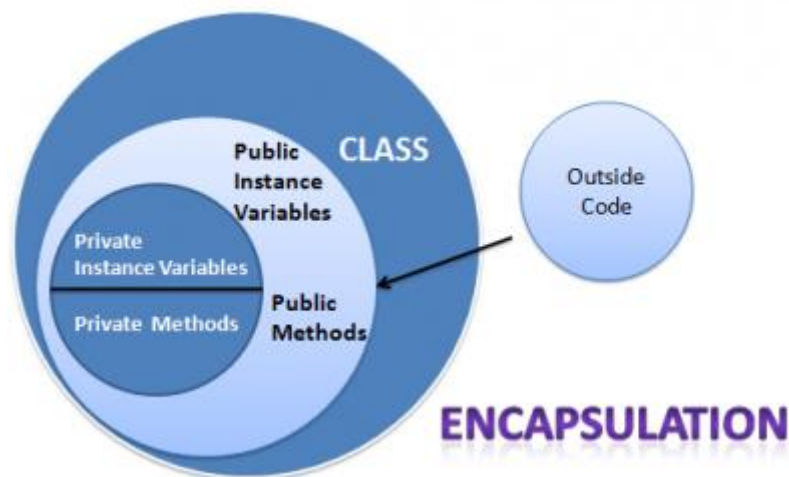
<pre> private String viTri; // Toán tử tạo lập số 1 (2) BongDen(){ this(40, true); System.out.println("Toán tử số 1"); } // Toán tử tạo lập không mặc định số 2 (3) BongDen(int w, boolean s){ this(w, s, "XX"); System.out.println("Toán tử số 2"); } // Toán tử tạo lập không mặc định số 3 (4) BongDen(int soWatts, Boolean batTat, String viTri){ this.soWatts = soWatts; this.batTat = batTat; this.viTri = new String(viTri); System.out.println("Toán tử số 3"); } } </pre>	<pre> private String viTri; // Toán tử tạo lập số 2 (2) BongDen(){ this(40, true); System.out.println("Toán tử số 1"); } // Toán tử tạo lập không mặc định số 2 (3) <i>BongDen(int w, boolean s){ this(w, s, "XX"); System.out.println("Toán tử số 2"); } // Toán tử tạo lập không mặc định số 3 (4) BongDen(int soWatts, Boolean batTat, String viTri){ this.soWatts = soWatts; this.batTat = batTat; this.viTri = new String(viTri); System.out.println("Toán tử số 3"); } } </i> </pre>
<pre> public class NhaKho{ public static void main(String args[]){ BongDen d1=new BongDen();// OK BongDen d2 = new BongDen(100, true, "Nha bep"); // OK BongDen d3 = new BongDen(100, true); //OK } } </pre>	<pre> class DenTuyp extends BongDen { private int doDai; private int mau; DenTuyp(int leng, int colo){ // (5) this(leng, colo, 100, true, "Chua biet"); } DenTuyp(int leng, int colo, int soWatt, boolean bt, String noi){ // (6) super(soWatt, bt, noi); this.doDai = leng; } } </pre>

	<pre> this.mau = colo; } } public class NhaKho{ public static void main(String args[]){ System.out.println("Tao ra bong đen tuyp"); DenTuyp d = new DenTuyp(20,5); } } </pre>
--	--

6.6. Bao bọc và che giấu thông tin

Tính đóng gói hay tính bao đóng (Encapsulation) là một trong bốn tính chất cơ bản của lập trình hướng đối tượng trong Java.

Tính đóng gói là kỹ thuật ẩn giấu thông tin không liên quan và hiện thị ra thông liên quan. Mục đích chính của đóng gói trong java là giảm thiểu mức độ phức tạp phát triển phần mềm.



Tính bao đóng trong Java là một tiến trình đóng gói code và dữ liệu lại với nhau vào trong một đơn vị unit đơn. Chúng ta có thể tạo một lớp được bao đóng hoàn toàn trong Java bằng việc tạo tất cả thành viên dữ liệu của lớp là private. Bây giờ, chúng ta sử dụng phương thức setter và getter để thiết lập và lấy dữ liệu trong nó.

Tính bao đóng là kỹ thuật tạo một trường của lớp private và cung cấp khả năng truy cập trường này qua các phương thức public. Nếu một trường được khai báo là private, nó không thể được truy cập bởi bên ngoài lớp, do đó có thể che dấu các trường có lớp này. Vì lý do này, tính bao đóng được ám chỉ như việc dấu dữ liệu (data hiding).

Để đạt được đóng gói trong Java chúng ta cần:

- Khai báo các thuộc tính của một lớp là private.
- Cung cấp phương thức setter và getter là public để có thể sửa đổi và xem các giá trị thuộc tính.

Ví dụ:

```
public class Student {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Các phương thức public setXXX() và getXXX() là các điểm truy cập đến các thuộc tính của lớp *Student*. Thông thường, các phương thức này được gọi là getters và setters. Vì vậy, bất kỳ đối tượng nào muốn truy cập vào các biến private sẽ truy cập chúng thông qua các trình getters và setters này.

Các thuộc tính của lớp *Student* có thể được truy cập như chương trình dưới đây:

```
public class EncapsulationExample {  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.setName("LTHĐT");  
        System.out.println(s.getName());  
    }  
}
```

}

Kết quả thực thi chương trình trên: LTHĐT

Quy ước đặt tên phương thức getter và setter như sau:

- **Getter**: bắt đầu bằng chữ **get** + viết hoa chữ đầu tiên tất cả các từ (viết hoa chữ đầu tiên của tên biến).
- **Setter**: bắt đầu bằng chữ **set** + viết hoa chữ đầu tiên tất cả các từ (viết hoa chữ đầu tiên của tên biến).

Lợi ích của đóng gói trong java

- Tất cả các trường (field) của lớp có có chế độ chỉ đọc (read-only) hoặc chỉ ghi (write-only), tức là chỉ có hàm getter hoặc setter.
- Một lớp có thể có toàn bộ điều khiển thông qua những gì được lưu giữ trong các trường (field) của nó.
- Người sử dụng của class không biết cách các class lưu trữ dữ liệu. Một class có thể thay đổi kiểu dữ liệu của một trường và người dùng class không cần sự thay đổi trong code.

6.7. Bài tập mẫu

Bài 1 : Tạo 1 lớp trừu tượng (abstract) Person chứa các thông tin về người, tạo tiếp 2 lớp Students và Teachers kế thừa lớp Person, tạo lớp Execute chứa hàm main để chạy chương trình.

```
import java.io.Console;
import java.util.Scanner;
abstract class Person {
    // các property hay state-thuộc tính của đối tượng
    String hoten;
    int age;
    String diachi;
    int luong;
    //các constructor
    Person() {
    }
    public Person(int age) {
        this.age = age;
    }
}
```

```

    }
    //cac method hay behavior-hanh vi cua doi tuong
    public void Nhap() {
        Console input = System.console();
        hoten = input.readLine("Nhap ho ten:");
        diachi = input.readLine("Nhap dia chi:");
    }
    public abstract void In();
    public abstract int Tinhluong();
}
class Students extends Person {
    int MaSV, Malop;
    public Students(int age) {
        super(age);
    }
    public Students() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma SV: ");
        MaSV = input.nextInt();
        System.out.println("Nhap Ma Lop: ");
        Malop = input.nextInt();
    }

    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
        System.out.println(MaSV);
        System.out.println(Malop);
    }
    public int Tinhluong() {
        return 150000;
    }
}
class Teachers extends Person {
    int Makhoa;
    public Teachers(int age) {
        super(age);
    }
    public Teachers() {
    }
    public void Nhap() {
        Scanner input = new Scanner(System.in);
        super.Nhap();
        System.out.println("Nhap Ma khoa: ");
        Makhoa = input.nextInt();
    }
}

```

```

    }
    public void In() {
        System.out.println(hoten);
        System.out.println(diachi);
        System.out.println(Makhoa);
    }
    public int Tinhluong() {
        return 500000;
    }
}
public class Execute {
    public static void main(String args[]) {
        Students st = new Students();
        st.Nhap();
        st.In();
        st.luong = st.Tinhluong();
        Teachers tc = new Teachers();
        tc.Nhap();
        tc.In();
        tc.luong = tc.Tinhluong();
    }
}

```

Bài 2 : **Shape** là lớp trừu tượng, trình triển khai của nó được cung cấp bởi lớp **Rectangle** và lớp **Circle**. Hai lớp này kế thừa lớp trừu tượng **Shape**.

// lop truu tuong Shape

```

abstract class Shape {
    abstract void draw();
}

```

//Trong tình huống này, trình triển khai được cung cấp bởi ai đó, ví dụ: người sử dụng cuối cùng nào đó

```

class Rectangle extends Shape {
    void draw() {
        System.out.println("Vẽ hình chữ nhật");
    }
}

```

```

class Circle1 extends Shape {
    void draw() {
        System.out.println("Vẽ hình tròn");
    }
}

```

//Trong tình huống này, phương thức được gọi bởi lập trình viên hoặc người dùng

```

class TestAbstraction1 {
    public static void main(String args[]) {
        Shape s = new Circle1(); //Trong tình huống này, đối tượng được cung cấp thông qua phương thức, chẳng hạn như getShape()
        s.draw();
    }
}

```

}

6.8. Bài tập thực hành

Bài 1: Tạo một lớp cha có tên là `Animal` với thuộc tính `name` và phương thức `makeSound()`. Tạo một lớp con `Dog` kế thừa lớp `Animal`, ghi đè phương thức `makeSound()` để in ra "Woof Woof".

Yêu cầu:

- Khai báo lớp `Animal` có phương thức `makeSound()`.
- Lớp `Dog` kế thừa `Animal` và ghi đè phương thức `makeSound()`.
- Tạo đối tượng của lớp `Dog` và gọi phương thức `makeSound()`.

Bài 2: Tạo lớp cha `Person` với thuộc tính `name` và phương thức `showInfo()`. Tạo lớp con `Employee` kế thừa từ `Person` và có thêm thuộc tính `salary`. Phương thức `showInfo()` của `Employee` phải gọi phương thức `showInfo()` của lớp cha bằng từ khóa `super`.

Yêu cầu:

- Lớp `Person` có phương thức `showInfo()` in ra tên của người.
- Lớp `Employee` kế thừa `Person` và ghi đè phương thức `showInfo()` để in thêm thông tin lương.
- Sử dụng từ khóa `super` để gọi phương thức của lớp cha.

Bài 3: Tạo một lớp `Shape` có phương thức `draw()`. Kế thừa lớp `Shape` tạo lớp `Rectangle`, và từ `Rectangle` tạo lớp `Square`. Ghi đè phương thức `draw()` ở cả hai lớp con `Rectangle` và `Square` để vẽ hình chữ nhật và hình vuông tương ứng.

Yêu cầu:

- Lớp `Shape` có phương thức `draw()`.
- Lớp `Rectangle` kế thừa `Shape` và ghi đè phương thức `draw()`.
- Lớp `Square` kế thừa `Rectangle` và ghi đè phương thức `draw()`.

Bài 4: Tạo lớp cha Vehicle có thuộc tính speed và phương thức showSpeed(). Tạo lớp con Car kế thừa từ Vehicle và thêm thuộc tính fuel. Ghi đè phương thức showSpeed() để in thêm thông tin về mức nhiên liệu.

Yêu cầu:

- Lớp Vehicle có thuộc tính speed và phương thức showSpeed().
- Lớp Car kế thừa Vehicle và ghi đè phương thức showSpeed() để in thêm thông tin về fuel.

Bài 5:

1) Xây dựng lớp **Ngươi** để quản lý một số thông tin gồm: họ và tên, năm sinh và các phương thức gồm: (3 điểm)

- + Một số toán tử tạo lập
- + Phương thức **nhapTT()** để nhập thông tin cho 1 người
- + Phương thức **hienThiTT()** để hiển thị thông tin của 1 người

2) Xây dựng lớp **SinhVien** kế thừa từ lớp Người, có thêm các thuộc tính bổ sung: lớp, điểm trung bình và các phương thức gồm:

- + Một số toán tử tạo lập
- + Viết đè phương thức **nhapTT()** để nhập thông tin cho 1 sinh viên
- + Viết đè phương thức **nhapTT()** để hiển thị thông tin của 1 sinh viên

3) Xây dựng lớp **QL_SinhVien** có các phương thức:

- + Nhập vào danh sách gồm m ($2 \leq m \leq 60$) sinh viên
- + Hiển thị danh sách các sinh viên có điểm trung bình lớn hơn 8.5